CREATION D’UN SITE WEB

TROUVE TON ARTISAN

Table des matières

1. Page de garde.

2. Contexte du projet.

   2.1 Présentation.
   2.2 Expressions des besoins.
   2.3 Contraintes.
   2.4 Livrables attendus.

3. Maquettes Figma.

   3.1 Description.
   3.2 Captures d'écran.
   3.3 Lien Figma.

4. Base de données.

   4.1 Modèle conceptuel de données (MCD).
   4.2 Modèle logique de données (MLD).
   4.3 Script SQL.

5. Sécurité.

   5.1 Validation des entrées.
   5.2 Gestion des erreurs.
   5.3 Protection des routes API.
   5.4 Sécurité base de données.
   5.5 Variables d'environnement.
   5.6 Cors.

6. Veille sécurité.

   6.1 Démarche.
   6.2 Vulnérabilités étudiées.

7. Repository GitHub et Site.

   7.1 Contenu dépôt.

8. Site en ligne.
   8.1 Hébergement possible.

1. **Page de garde.**

Développement d'une application web
de mise en relation avec des artisans

Nom : Châtelier
Prénom : Jimmy

Formation : Développeur Web
Établissement : CEF (Centre Européen de Formation)

Date : 20/03/2026

Technologies utilisées :

Frontend :
- React.js
- Bootstrap
- Sass

Backend :
- Node.js
- Express

Base de données :
- MySQL
- Sequelize

Versionning :
- Git / GitHub

Projet réalisé dans le cadre du devoir :
« Créez le site Web Trouve ton artisan avec React.js »

DOSSIER : TROUVE TON ARTISAN

---

2. Contexte du projet

2.1 Présentation de l'entreprise

Le projet « **Trouve ton artisan** » a pour objectif de développer une application web permettant de mettre en relation des particuliers avec des artisans qualifiés.

Dans un contexte où la recherche de professionnels fiables peut s'avérer complexe, cette plateforme vise à simplifier l'accès à l'information en proposant une interface intuitive et centralisée (catégorie, spécialité, localisation).

Ce type de service est particulièrement utile dans une région comme **Auvergne-Rhône-Alpes**, où la demande en artisans est importante.

2.2 Expression des besoins

Le client souhaite :

- Une application web accessible et responsive
- Une recherche d'artisans par :
  - Catégorie.
  - Spécialité.
  - Nom.
- Une visualisation des artisans avec :
  - Nom.
  - Note.
  - Ville.
  - Description.
- Une fiche détaillée par artisan
- Un formulaire de contact
- Une interface simple et intuitive

L'application doit également être responsive afin d'être utilisable sur ordinateur, tablette et mobile.

2.3 Contraintes

- Application responsive (Desktop / Tablet / Mobile).
- Utilisation de :

    - React.js (frontend).
    - Node.js + Express (backend).
    - MySQL (base de données).
- Respect des bonnes pratiques web (UX, sécurité).
- Versionnement via Git / GitHub.

2.4 Livrables attendus

- Application web fonctionnelle.
- Base de données complète.
- Maquettes Figma.
- Documentation technique.
- Dépôt GitHub.
- Site accessible en ligne.

---

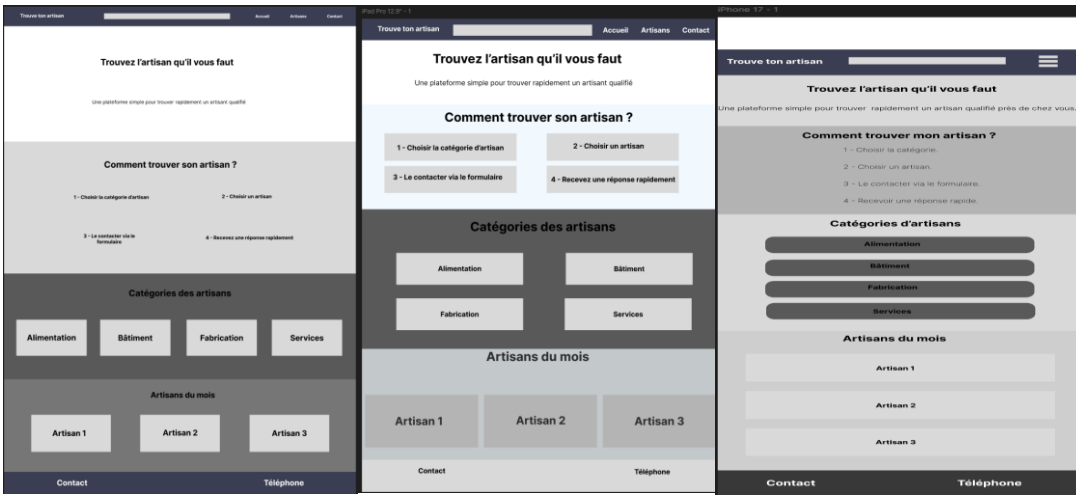
3. Maquettes Figma

3.1 Description

Les maquettes ont été réalisées avec Figma pour les 3 formats :

- Desktop.
- Tablette.
- Mobile.

3.2 Captures d'écran



- maquette-desktop.png
- maquette-tablet.png
- maquette-mobile.png

3.3 Lien Figma

https://www.figma.com/files/team/1575205544839041032/recents-and-sharing?fuid=1575205540830509266

---

4. Base de données

4.1 Modèle Conceptuel de Données (MCD)

La base de données a été conçue selon une architecture relationnelle, permettant d'assurer la cohérence et l'intégrité des données.

Elle repose sur trois entités principales :

- **Catégorie.**
- **Spécialité.**
- **Artisan.**

Chaque catégorie regroupe plusieurs spécialités, et chaque spécialité est associée à plusieurs artisans.

Cette organisation permet une structuration logique des données et facilite les opérations de filtrage et de recherche.

Relations :

- Une catégorie possède plusieurs spécialités.
- Une spécialité possède plusieurs artisans.

4.2 Modèle Logique de Données (MLD)

Table categories :

- id
- name
- slug

Table specialties :

- id
- name
- slug
- categoryId

Table artisans

- id
- name
- slug
- about
- email
- website
- city
- rating
- isTop
- specialtyId

4.3 SCRIPT SQL

1. Fichier sql/01_create_database.sql
2. Fichier sql/02_seed_database.sql
3. Fichier sql/03_verification.sql

- Création de la base de données.

```
DROP DATABASE IF EXISTS trouve_ton_artisan;
CREATE DATABASE trouve_ton_artisan;
USE trouve_ton_artisan;
```

- Table : categories.

```
CREATE TABLE categories (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
slug VARCHAR(255) NOT NULL UNIQUE,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

- Table : specialties.

```
CREATE TABLE specialties (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
slug VARCHAR(255) NOT NULL,
categoryId INT NOT NULL,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

CONSTRAINT fk_specialties_category
FOREIGN KEY (categoryId)
REFERENCES categories(id)
ON DELETE CASCADE
);
```

- Table : artisans.

```
CREATE TABLE artisans (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
slug VARCHAR(255) NOT NULL UNIQUE,
about TEXT,
email VARCHAR(255),
website VARCHAR(255),
city VARCHAR(255),
rating FLOAT DEFAULT 0,
isTop BOOLEAN DEFAULT FALSE,
specialtyId INT NOT NULL,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

CONSTRAINT fk_artisans_specialty
FOREIGN KEY (specialtyId)
REFERENCES specialties(id)
ON DELETE CASCADE
);
```

- Table : contacts (formulaire).

```
CREATE TABLE contacts (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
email VARCHAR(255) NOT NULL,
message TEXT NOT NULL,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

- Index pour améliorer les performances.

```
CREATE INDEX idx_categories_slug ON categories(slug);
CREATE INDEX idx_specialties_slug ON specialties(slug);
CREATE INDEX idx_artisans_slug ON artisans(slug);
```

---

5. Sécurité

Plusieurs mesures ont été mises en place afin de garantir la sécurité de l'application :

5.1 Validation des entrées

Les données saisies par l'utilisateur sont validées afin de garantir leur conformité (format, présence des champs obligatoires) :

- Champs obligatoires.
- Format email.
- Longueur des champs.

Empêche les données invalides ou malveillantes.

5.2 Gestion des erreurs

Les erreurs sont traitées de manière contrôlée afin d'éviter l'exposition d'informations sensibles.

- try/catch dans les routes.
- Messages d'erreur contrôlés.

Évite l'exposition d'informations sensibles.

5.3 Protection des routes API

- Routes organisées.
- Accès contrôlé côté serveur.

5.4 Sécurité base de données

L'utilisation de l'ORM Sequelize permet de sécuriser les requêtes vers la base de données et de prévenir les injections SQL.

- Utilisation de Sequelize (ORM).
- Prévention des injections SQL.

Très important : les requêtes sont sécurisées automatiquement.

5.5 Variables d'environnement

Les informations sensibles (configuration, accès base de données) sont stockées dans des fichiers .env, non exposés dans le code.

- Utilisation de .env

Permet de protéger :

- Accès base de données.
- Configuration serveur.

5.6 CORS

- Configuration du backend pour autoriser uniquement le frontend.

---

6. Veille sur la sécurité

6.1 Démarche

Une veille technologique a été réalisée sur les principales vulnérabilités web, notamment celles recensées dans l'OWASP Top 10.

Les risques suivants ont été étudiés :

- Les injections SQL.
- Les attaques XSS.
- Les mauvaises validations des entrées.

Des mesures adaptées ont été mises en place pour limiter ces risques, notamment grâce à l'utilisation de frameworks modernes et à la validation des données.

Une attention particulière a été portée à la structuration du code, à la réutilisabilité des composants et au respect des bonnes pratiques de développement.

6.2 Vulnérabilités étudiées

Injection SQL

Solution :

- Utilisation de Sequelize.

XSS (Cross-Site Scripting)

Solution :

- React protège automatiquement contre l'injection HTML.

Mauvaise validation des données

Solution :

- Validation côté client et serveur.

Améliorations possibles

- Authentification utilisateur.
- Protection JWT.
- Rate limiting.

---

7. Repository GitHub

https://github.com/chatelierjimmy-max/trouve-ton-artisan

7.1 Contenu du dépôt

- Code frontend React.
- Code backend Node.js.
- Script SQL création.

- Script SQL import données.
- Dossier docs.

---

8. Site en ligne

Allez sur :

https://www.Trouve-ton-artisan.com

8.1 Hébergement possible

- Vercel (frontend).
- Render / Railway (backend).

9.Conclusion

Le projet **Trouve ton artisan** permet de répondre efficacement au besoin de mise en relation entre particuliers et professionnels.

Les objectifs ont été atteints :

- Application fonctionnelle.
- Interface responsive.
- Base de données structurée.
- Bonnes pratiques de développement respectées.